ISO615 · Desarrollo de Software con TEC-SEM

Entregable 1: Diseño y Configuración del Entorno

Documento de análisis y diseño (Script DB / Coding)

Sistema: Agua Clara RD

Dato	Detalle
Proyecto	Agua Clara RD: reporte, seguimiento y gestión operativa de averías y falta de agua (República Dominicana)
Entregable	1 · Requerimientos, tecnologías, arquitectura y modelo de datos
Fecha	5 de noviembre de 2026
Equipo	[Escribe aquí los nombres de los integrantes del grupo]
Repositorio	[URL del repositorio en GitHub]
Versión del sistema	v3.0 (Épica 1 y Épica 2 implementadas)

Contenido

1. Introducción
2. Análisis de requerimientos
3. Selección de tecnologías
4. Diseño de la arquitectura
5. Modelo de datos
6. Script de base de datos
7. Configuración del entorno
8. Conclusiones y limitaciones
Anexo A. Catálogo de endpoints de la API
Anexo B. Cómo cumple este documento la rúbrica

1. Introducción

Agua Clara RD es una aplicación web para que las personas reporten averías o falta de agua en su barrio y consulten el estado de su reporte, y para que el equipo operativo convierta esos reportes en trabajos controlados: los asigna, los aprueba, vigila sus plazos, registra incidencias y genera reportes con trazabilidad completa.

Este documento establece las bases del sistema: qué debe hacer (requerimientos), con qué se construye (tecnologías), cómo está organizado (arquitectura del front-end, del back-end y de la interacción entre componentes) y cómo se guardan los datos (modelo relacional y script de base de datos). **El diseño no es solo teórico:** ya está implementado y probado (63 pruebas automáticas), y los diagramas y el script se generaron a partir del sistema real para que coincidan con él.

1.1 Alcance

Parte del sistema	Qué cubre
Parte pública	Reportar una avería, recibir un código, consultar el seguimiento y ver consejos de ahorro, sin crear cuenta.
Épica 1 · Usuarios y accesos	Registro, inicio y cierre de sesión, roles, permisos, bloqueo, recuperación de contraseña, perfil y auditoría de accesos.
Épica 2 · Gestión operativa	Panel de operaciones, ciclo de vida, asignaciones, aprobaciones, alertas, incidencias, reportes y bitácora inmutable.

1.2 Roles de la plataforma

Rol	Responsabilidad
Ciudadano (sin cuenta)	Reporta averías y consulta el estado con su código.
Cliente	Usuario registrado sin acceso a las funciones operativas.

Rol	Responsabilidad
Agente, Arquitecto, Ingeniero	Operadores: registran y siguen las operaciones que crean o tienen asignadas.
Supervisor	Crea, asigna, aprueba, configura plazos y revisa la bitácora.
Gerente	Genera y exporta reportes de operaciones.
Auditor	Consulta la bitácora y los reportes, en solo lectura.
Administrador	Gestiona usuarios, roles y operaciones (no modifica permisos).
Super Administrador	Control total, incluida la matriz de permisos.

2. Análisis de requerimientos

2.1 Requerimientos funcionales

Prioridad: **Alta** = el sistema no cumple su objetivo sin ello; **Media** = mejora importante. “Origen” indica la épica o parte del sistema donde se implementó.

ID	Requerimiento	Prioridad	Origen
RF-01	Reportar una avería o falta de agua indicando municipio, sector, tipo, descripción y contacto opcional, sin crear cuenta.	Alta	Pública
RF-02	Generar un código de seguimiento al registrar un reporte.	Alta	Pública
RF-03	Consultar el estado de un reporte con su código, con mensaje claro si el código no existe.	Alta	Pública
RF-04	Mostrar consejos de ahorro de agua y enlaces oficiales (CAASD e INAPA).	Media	Pública
RF-05	No mostrar públicamente los datos de contacto del ciudadano.	Alta	Pública
RF-06	Registrar usuarios (por un administrador y por autorregistro con rol Cliente) con correo único.	Alta	Épica 1
RF-07	Iniciar y cerrar sesión, registrando fecha, hora, IP y localización.	Alta	Épica 1
RF-08	Gestionar roles: crear, modificar, desactivar y eliminar, con reglas que protegen roles en uso.	Alta	Épica 1
RF-09	Gestionar permisos por rol y módulo (crear, consultar, modificar, eliminar) con efecto inmediato.	Alta	Épica 1
RF-10	Restringir las funcionalidades según el perfil: ocultar, deshabilitar o denegar el acceso.	Alta	Épica 1
RF-11	Recuperar la contraseña con un enlace temporal y de un solo uso.	Media	Épica 1
RF-12	Bloquear y reactivar usuarios, con motivo obligatorio.	Media	Épica 1
RF-13	Administrar el perfil propio: datos, fotografía y contraseña.	Media	Épica 1

ID	Requerimiento	Prioridad	Origen
RF-14	Consultar el historial de accesos con filtros.	Media	Épica 1
RF-15	Mostrar un panel con operaciones creadas, en proceso, completadas, rechazadas y vencidas, con filtros y fecha de última actualización.	Alta	Épica 2
RF-16	Registrar operaciones con código único, tipo, descripción, responsable y fecha límite.	Alta	Épica 2
RF-17	Controlar el ciclo de vida de la operación con transiciones permitidas y validaciones de negocio.	Alta	Épica 2
RF-18	Asignar y reasignar responsables autorizados, con motivo y notificación.	Alta	Épica 2
RF-19	Configurar y ejecutar flujos de aprobación por tipo: aprobar, devolver o rechazar con comentario.	Alta	Épica 2
RF-20	Configurar plazos por tipo y generar alertas de operaciones por vencer y vencidas, sin duplicados.	Media	Épica 2
RF-21	Registrar incidencias vinculadas a una operación, con comentarios y resolución.	Alta	Épica 2
RF-22	Generar reportes operativos con métricas definidas y exportarlos a CSV según el permiso.	Media	Épica 2
RF-23	Mantener una bitácora inmutable de cambios con valores anteriores y nuevos, consultable con filtros.	Alta	Épica 2

2.2 Requerimientos no funcionales

ID	Categoría	Requerimiento y meta	Cómo se verifica
RNF-01	Seguridad · autenticación	Contraseñas cifradas con bcrypt; sesión en cookie HttpOnly de 8 horas; bloqueo de 10 minutos tras 5 intentos fallidos.	Pruebas “Contraseñas cifradas” e “Intentos repetidos”; tabla usuarios.
RNF-02	Seguridad · autorización	Toda ruta privada valida sesión y permiso en el servidor (401 sin sesión, 403 sin permiso), aunque se altere la página.	Pruebas de acceso autorizado y no autorizado por rol.
RNF-03	Seguridad · entradas	Consultas SQL parametrizadas, validación en el servidor, límite de 400 KB por petición y neutralización de fórmulas en el CSV.	Prueba de inyección SQL y de exportación CSV.
RNF-04	Integridad	Llaves foráneas activas, restricciones UNIQUE y NOT NULL, transacciones en operaciones de varias tablas y bitácora protegida por triggers.	Script de BD; prueba “La bitácora no se puede modificar ni eliminar”.
RNF-05	Trazabilidad	Toda acción relevante deja registro con actor, fecha y hora, acción, resultado y valores antes y después.	Pestañas Auditoría y Bitácora; tablas auditoria y bitacora.

ID	Categoría	Requerimiento y meta	Cómo se verifica
RNF-06	Rendimiento	Meta: consultas principales por debajo de 1 segundo. Medido con 5 006 operaciones y 20 018 registros de bitácora: listado 40 ms (p95 51.6 ms), reporte 46 ms (p95 60 ms), bitácora 3.6 ms; el login tarda ≈ 103 ms por el cifrado bcrypt.	Prueba de carga propia (30 peticiones por consulta).
RNF-07	Usabilidad	Interfaz en español, mensajes de error claros, formularios con etiquetas, foco visible y modo claro y oscuro.	Recorrido de demostración y capturas de evidencia.
RNF-08	Compatibilidad y adaptación	Funciona en computadora y teléfono (sin desbordes desde 390 px de ancho) y en navegadores modernos.	Capturas en versión móvil.
RNF-09	Portabilidad	Se instala con tres comandos; la base de datos es un solo archivo; requiere Node.js 18 o superior.	Instrucciones del apartado 7, probadas en una copia limpia.
RNF-10	Mantenibilidad	Código modular, 63 pruebas automáticas, esquema versionado y migración automática al iniciar.	Comando npm test; prueba de migración.
RNF-11	Escalabilidad	Adecuado para cientos de usuarios y miles de operaciones (medido). Con mayor carga se migra a PostgreSQL o MySQL cambiando solo la capa de datos.	Medición de RNF-06; apartado 3.3.
RNF-12	Privacidad	El contacto del ciudadano es opcional y no se muestra; las contraseñas nunca se guardan ni se registran.	Prueba de que la bitácora no expone secretos.
RNF-13	Respaldo y disponibilidad	La copia de seguridad es copiar el archivo .db. Las alertas automáticas requieren el servidor encendido (limitación conocida).	Apartado 8.
RNF-14	Localización	Textos, fechas y mensajes en español de República Dominicana (es-DO).	Interfaz del sistema.

2.3 Reglas de negocio principales

#	Regla
1	Una operación no se inicia sin responsable y solo sigue las transiciones del ciclo de vida (figura 8).
2	Un tipo con aprobación obligatoria no se completa sin la decisión del rol aprobador; quien tiene asignada la operación no puede aprobarla.
3	Cancelar, reabrir, rechazar, devolver y reasignar exigen un motivo; cerrar exige el resultado.
4	No se cierra una operación con incidencias sin resolver.
5	Solo pueden ser responsables usuarios activos cuyo rol tenga permiso operativo.

#	Regla
6	Un operador solo modifica operaciones propias; supervisión modifica todas.
7	La bitácora no se puede modificar ni eliminar.

3. Selección de tecnologías

3.1 Criterios de selección

- **Facilidad de instalación y demostración:** que el sistema funcione en cualquier computadora del grupo con pocos pasos.
- **Costo:** preferir tecnologías gratuitas y de código abierto.
- **Seguridad y madurez:** librerías muy usadas y mantenidas.
- **Conocimiento del equipo:** un solo lenguaje (JavaScript) en toda la aplicación.
- **Camino de crecimiento:** poder migrar a servicios más grandes sin rehacer el sistema.

3.2 Tecnologías de código abierto

Tecnología	Versión	Licencia	Uso en el sistema	Alternativas consideradas
Node.js	22	MIT	Ejecuta el servidor (back-end) y las pruebas.	Python (Flask/Django), Java (Spring)
Express	4.22	MIT	Framework de la API REST y servidor de archivos estáticos.	Fastify, Koa
SQLite	3	Dominio público	Base de datos relacional en un archivo.	PostgreSQL, MySQL, MongoDB
better-sqlite3	12.11	MIT	Acceso a SQLite desde Node, con consultas parametrizadas.	node-sqlite3, Prisma
bcryptjs	3.0	BSD-3-Clause	Cifrado de contraseñas (hash con sal).	argon2, scrypt
HTML5, CSS3, JavaScript	ES2022	Estándares abiertos	Front-end sin framework: menos dependencias y carga rápida.	React, Vue, Angular
node:test	incluido	MIT	Pruebas automáticas de la API (63 casos).	Jest, Mocha
Git	2.x	GPL-2.0	Control de versiones y trabajo por ramas.	Mercurial
DB Browser for SQLite	3.13	GPL-3.0 / MPL-2.0	Inspección visual de la base de datos.	DBeaver

Tecnología	Versión	Licencia	Uso en el sistema	Alternativas consideradas
Graphviz y Mermaid	—	EPL / MIT	Diagramas de este documento, generados desde código.	draw.io, Lucidchart
Playwright	1.56	Apache-2.0	Pruebas en navegador, capturas y video de evidencia.	Selenium, Cypress
geoip-lite (opcional)	1.4	Apache-2.0	Localización por IP en la auditoría.	Servicios de pago (ver 3.3)

3.3 Tecnologías y servicios propietarios

El sistema se diseñó para depender lo menos posible de servicios de pago. Estos son los servicios propietarios que se usan y los que se evaluaron para la siguiente fase:

Servicio	Estado	Para qué sirve	Decisión y motivo
GitHub (Microsoft)	En uso	Repositorio remoto, historial de cambios y entrega del código.	Estándar del curso; plan gratuito suficiente.
Google Fonts (Google)	En uso	Entrega de la tipografía Inter (la fuente es libre; el servicio es de Google). Hay tipografías de respaldo si no carga.	Opcional: si falla, la página sigue funcionando.
Correo transaccional: SendGrid, Amazon SES o Gmail API	Evaluado	Enviar el enlace de recuperación de contraseña y las alertas por correo.	Pendiente: requiere cuenta y credenciales. Hoy el correo es simulado (MODO_DEMO).
Mensajería: Twilio (SMS / WhatsApp)	Evaluado	Alertas a los responsables fuera de la página.	Pendiente: tiene costo por mensaje.
Geolocalización de IP: MaxMind GeoIP2 o ipinfo.io	Evaluado	Ciudad y país reales en la auditoría de accesos.	Se usa la opción gratuita geoip-lite; la comercial ofrece más precisión.
Mapas: Google Maps Platform	Evaluado	Ubicar la avería en un mapa.	Pendiente: por privacidad solo se pide una ubicación general.
Alojamiento: Render, Railway o Azure	Evaluado	Publicar el sistema en internet con HTTPS.	Pendiente; el sistema ya soporta cookie segura y proxy (COOKIE_SECURE, TRUST_PROXY).

Nota para el equipo: confirmen cuáles servicios propietarios van a usar de verdad. Hoy solo GitHub y Google Fonts están en uso; el resto son candidatos documentados con su justificación.

3.4 Decisión de la base de datos

✔ cumple · ◐ cumple en parte · ✖ no cumple. Las valoraciones son del equipo, con base en el contexto del proyecto.

Criterio	SQLite	PostgreSQL	MySQL	MongoDB Atlas	Firebase
Instalar sin servidor aparte	✔	✖	✖	◐ (nube)	◐ (nube)
Integridad relacional (llaves, transacciones)	✔	✔	✔	✖	✖
Costo para el proyecto	✔	✔	✔	◐	◐
Portabilidad (un archivo)	✔	✖	✖	✖	✖
Muchos usuarios simultáneos	◐	✔	✔	✔	✔
Bitácora inmutable con triggers	✔	✔	✔	✖	✖

Decisión: SQLite. El modelo es claramente relacional (usuarios, roles, permisos, operaciones, aprobaciones), la demostración debe poder ejecutarse en cualquier computadora y los volúmenes medidos (miles de operaciones) responden en milisegundos. Si el sistema crece, se migra a **PostgreSQL**: el esquema es estándar y solo cambia `server/db.js` y la sintaxis de algunas consultas.

4. Diseño de la arquitectura

4.1 Visión general

El sistema sigue una arquitectura **cliente–servidor en tres capas**: presentación (la página en el navegador), aplicación (el servidor Node.js con la API REST y las reglas de negocio) y datos (SQLite). La página nunca habla con la base de datos: todo pasa por la API, que valida la sesión y los permisos. Esto evita exponer credenciales y permite que las reglas de seguridad no puedan saltarse desde el navegador.

Figura 1. Arquitectura en tres capas

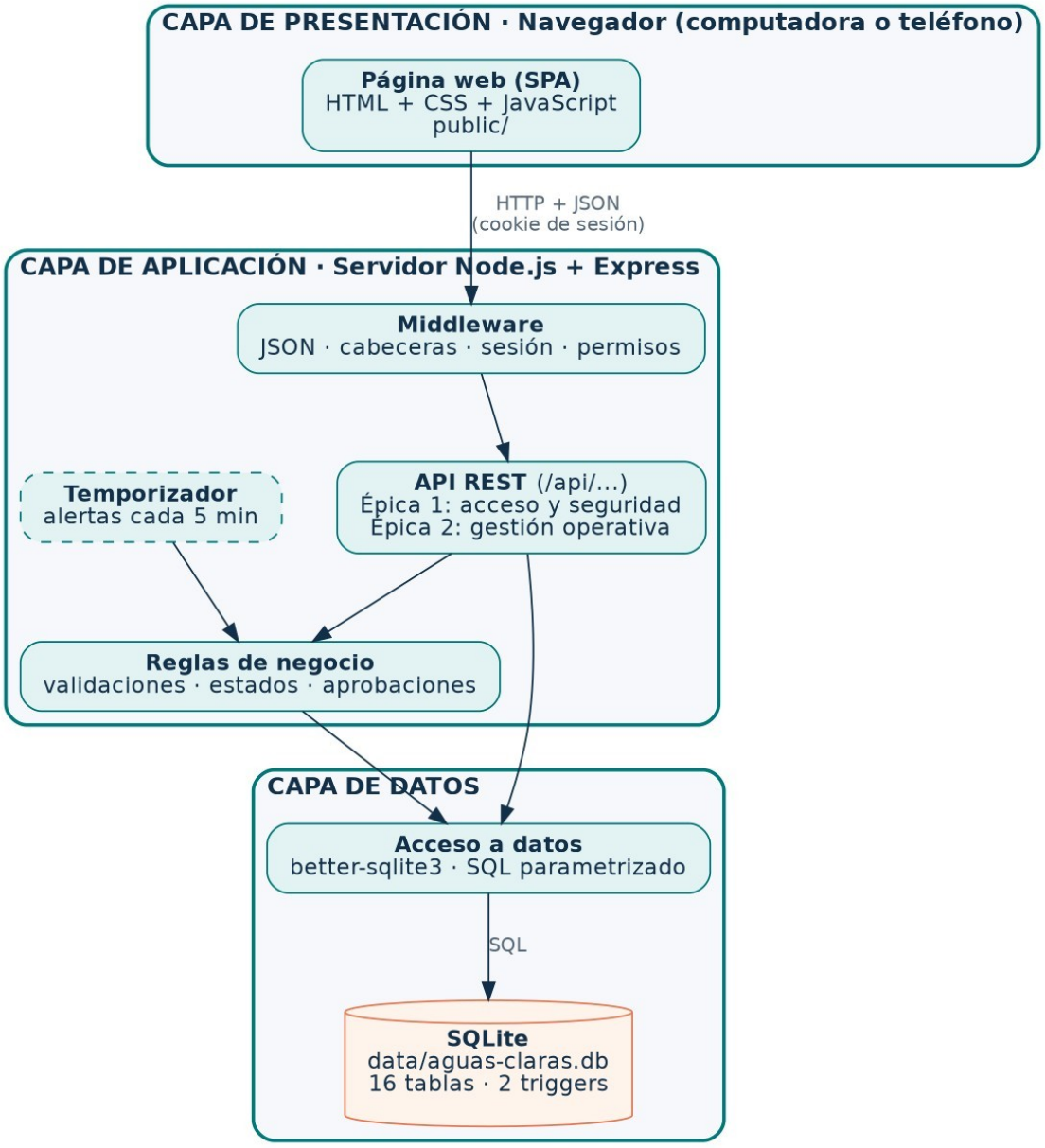
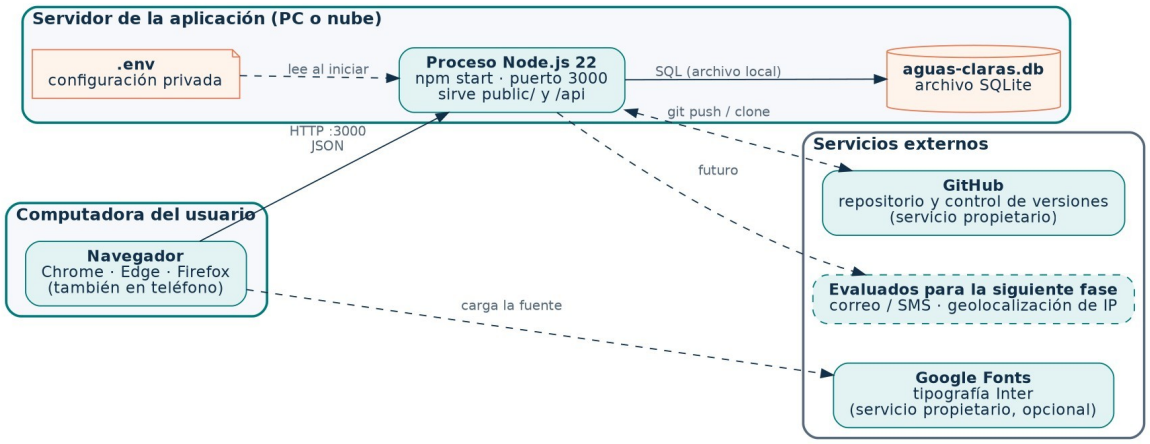


Figura 2. Despliegue y servicios externos



Una sola computadora o servidor ejecuta el proceso Node.js, que lee su configuración de .env y guarda los datos en el archivo SQLite. GitHub guarda el código. Los servicios evaluados aparecen con línea punteada.

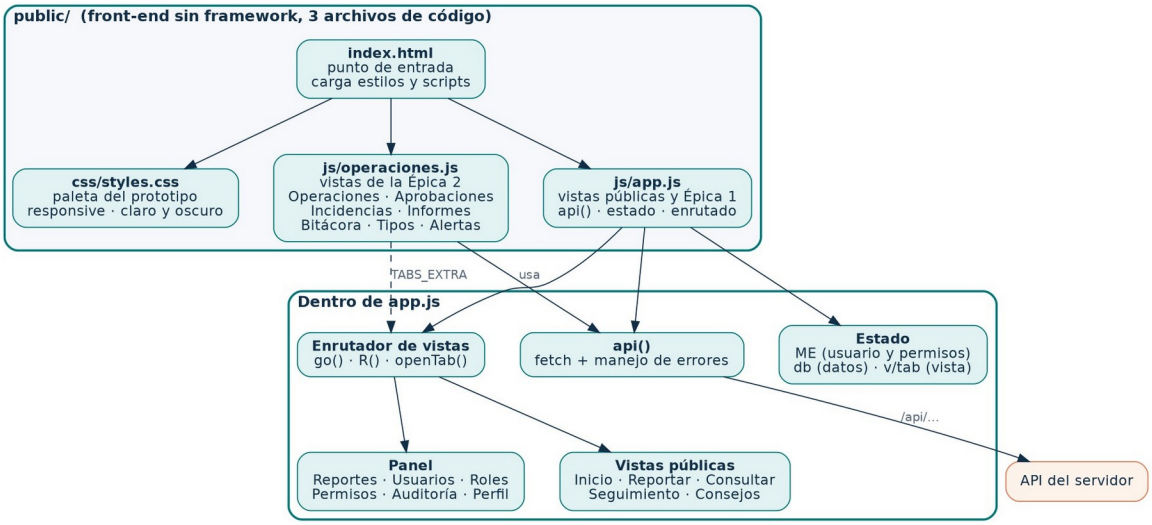
4.2 Estructura del front-end

El front-end es una **aplicación de una sola página (SPA) sin framework**: HTML, CSS y JavaScript estándar. Esta decisión reduce dependencias, evita un paso de compilación y hace que la página cargue rápido. Su estructura es:

Archivo	Responsabilidad
`public/index.html`	Punto de entrada: define el contenedor y carga los estilos y los scripts.
`public/css/styles.css`	Estilos con la paleta del prototipo, diseño adaptable (computadora y teléfono), modo claro y oscuro y foco visible.
`public/js/app.js`	Núcleo: función api() para llamar al servidor, estado (usuario y permisos, datos), enrutador de vistas y las pantallas públicas y de la Épica 1.
`public/js/operaciones.js`	Pantallas de la Épica 2. Se registran en el panel con el mecanismo TABS_EXTRA, sin modificar el núcleo.

- **Estado:** ME guarda el usuario y su matriz de permisos; db guarda los datos cargados; v y tab indican la vista actual.
- **Enrutado:** go() cambia de pantalla, openTab() abre una pestaña del panel y R() vuelve a dibujar.
- **Interfaz guiada por permisos:** cada botón se habilita según ME.perms; lo que el rol no puede hacer aparece deshabilitado o con candado, y el servidor lo vuelve a validar.
- **Comunicación:** toda acción llama a api(), que envía JSON con la cookie de sesión y devuelve el resultado o el mensaje de error del servidor.
- **Extensión:** una épica nueva solo agrega su archivo de vistas y las registra en TABS_EXTRA (así se agregó la Épica 2).

Figura 3. Estructura del front-end



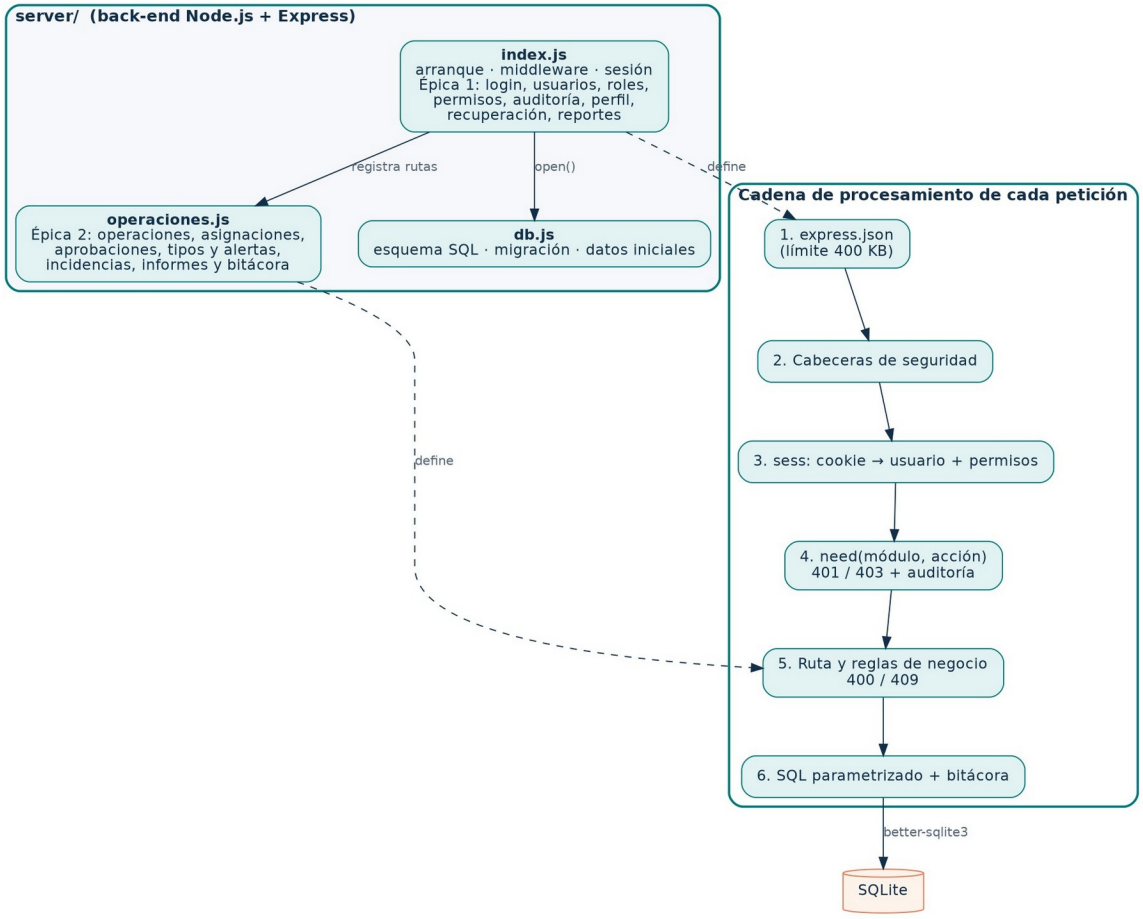
4.3 Estructura del back-end

El back-end es un servidor **Node.js con Express** organizado en tres módulos y una cadena de procesamiento fija para cada petición:

Archivo	Responsabilidad
`server/index.js`	Arranque, middleware, sesión y permisos. Rutas de la parte pública y de la Épica 1 (login, usuarios, roles, permisos, auditoría, perfil, recuperación, reportes).
`server/operaciones.js`	Rutas y reglas de negocio de la Épica 2: operaciones, asignaciones, aprobaciones, tipos y alertas, incidencias, informes y bitácora.
`server/db.js`	Esquema SQL, migración automática, datos iniciales y de demostración.

- **Seguridad por capas:** primero se identifica la sesión (cookie → usuario y permisos), luego need(módulo, acción) autoriza; si no hay sesión responde 401 y si no hay permiso responde 403 y lo registra.
- **Reglas de negocio en el servidor:** cada ruta valida los datos y el estado antes de escribir; las violaciones responden 400 o 409 con un mensaje en español y quedan en la bitácora como “Rechazado”.
- **Transacciones:** las acciones que tocan varias tablas (crear operación, asignar, aprobar) se ejecutan en una transacción.
- **Tareas automáticas:** un temporizador revisa los vencimientos cada 5 minutos y genera alertas sin duplicados.
- **Convenciones REST:** JSON, códigos HTTP significativos (200, 201, 400, 401, 403, 404, 409, 429) y rutas bajo /api. El catálogo completo está en el anexo A.

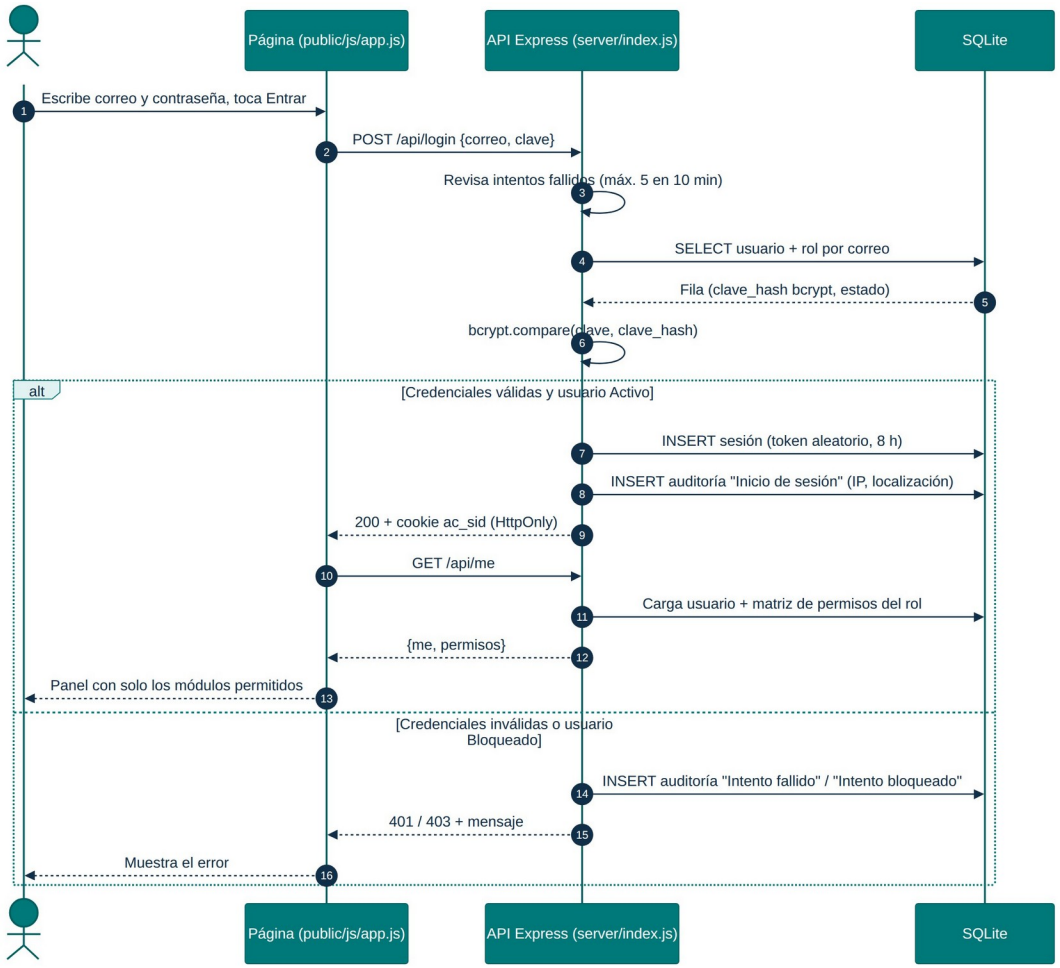
Figura 4. Estructura del back-end y cadena de procesamiento de una petición



4.4 Modelo de interacción entre componentes

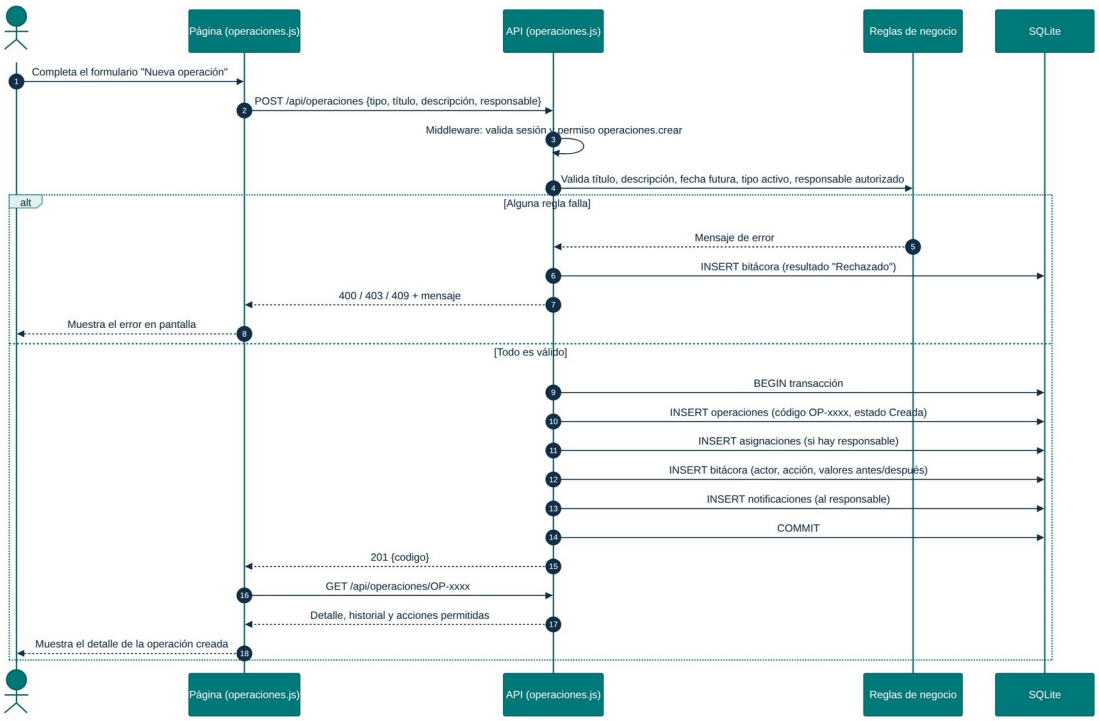
Estos diagramas muestran cómo colaboran la página, la API, las reglas de negocio y la base de datos en los flujos más importantes.

Figura 5. Interacción: inicio de sesión



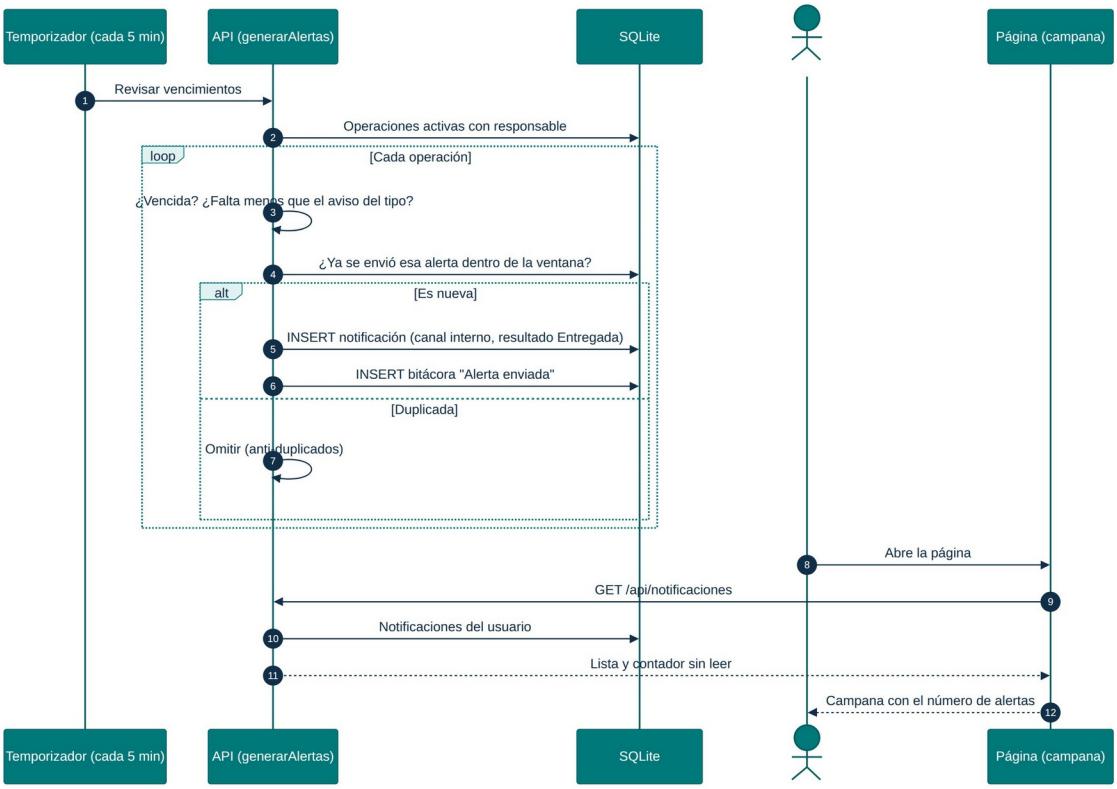
Muestra la seguridad: límite de intentos, comparación con bcrypt, sesión en cookie HttpOnly y registro en la auditoría, tanto si el acceso es válido como si falla.

Figura 6. Interacción: crear una operación



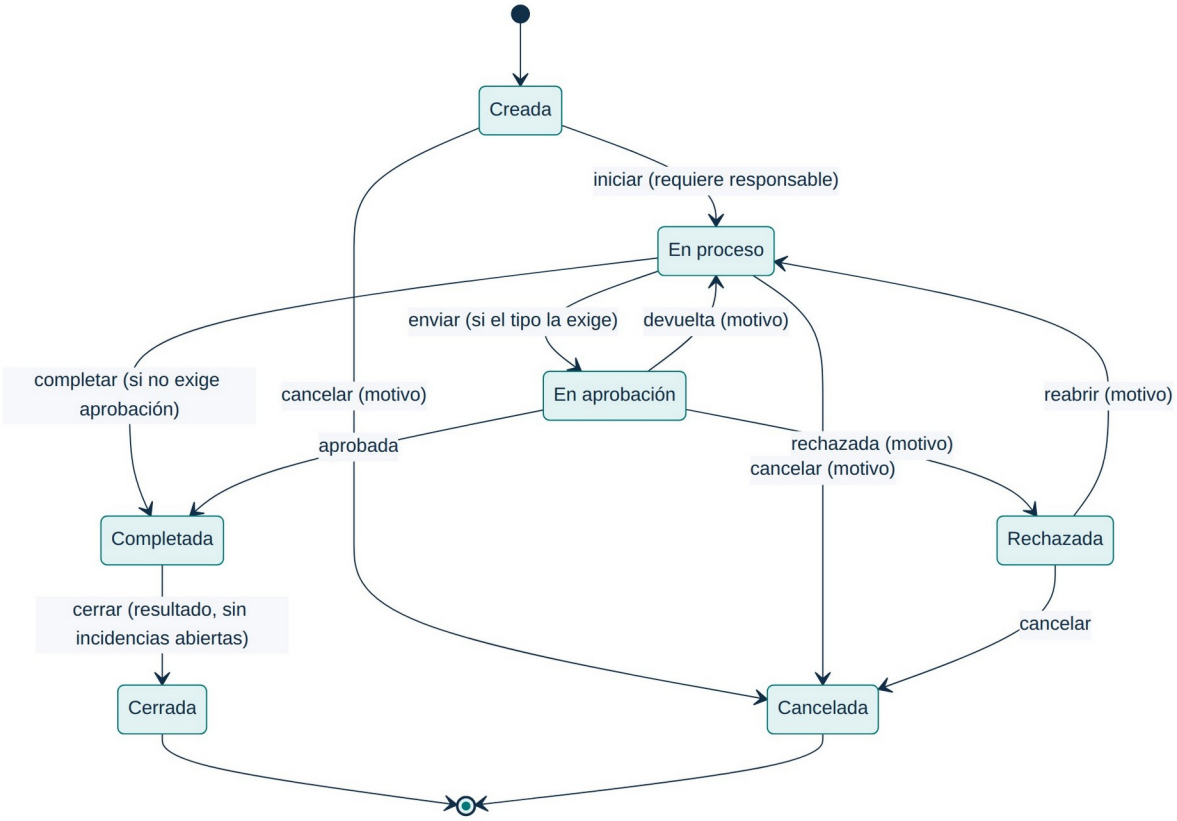
Muestra la validación en dos niveles (permiso y reglas de negocio), el rechazo registrado en la bitácora y la transacción que guarda la operación, la asignación, la bitácora y la notificación.

Figura 7. Interacción: alertas automáticas



El temporizador revisa los vencimientos, evita duplicados dentro de la ventana configurada y deja las notificaciones disponibles para la campana de la página.

Figura 8. Ciclo de vida de una operación (diagrama de estados)



Define las únicas transiciones permitidas. Cancelada y Cerrada son estados finales.

4.5 Contrato de interacción

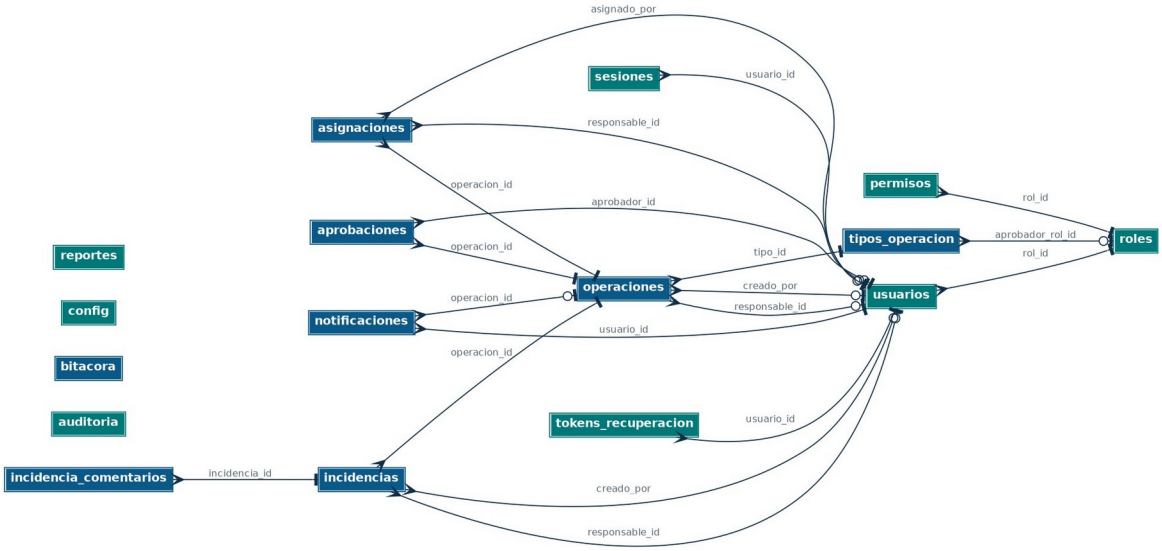
Aspecto	Definición
Formato	JSON en ambos sentidos; el CSV de informes es la única excepción.
Sesión	Cookie ac_sid (HttpOnly, SameSite=Lax; Secure cuando hay HTTPS), vigencia de 8 horas.
Errores	{ "error": "mensaje en español" } con el código HTTP correspondiente.
Permisos	Cada ruta privada declara su módulo y acción (ver anexo A); la matriz vive en la tabla permisos.
Fechas	Milisegundos Unix en la base de datos; la página las muestra en formato es-DO.

5. Modelo de datos

El modelo relacional tiene **16 tablas**, **19 llaves foráneas**, **5 índices** y **2 triggers**. Los diagramas se generaron directamente desde la base de datos real, así que no pueden diferir de ella. Notación: la pata de gallo indica el lado “muchos”; la barra, relación obligatoria; el círculo, relación opcional.

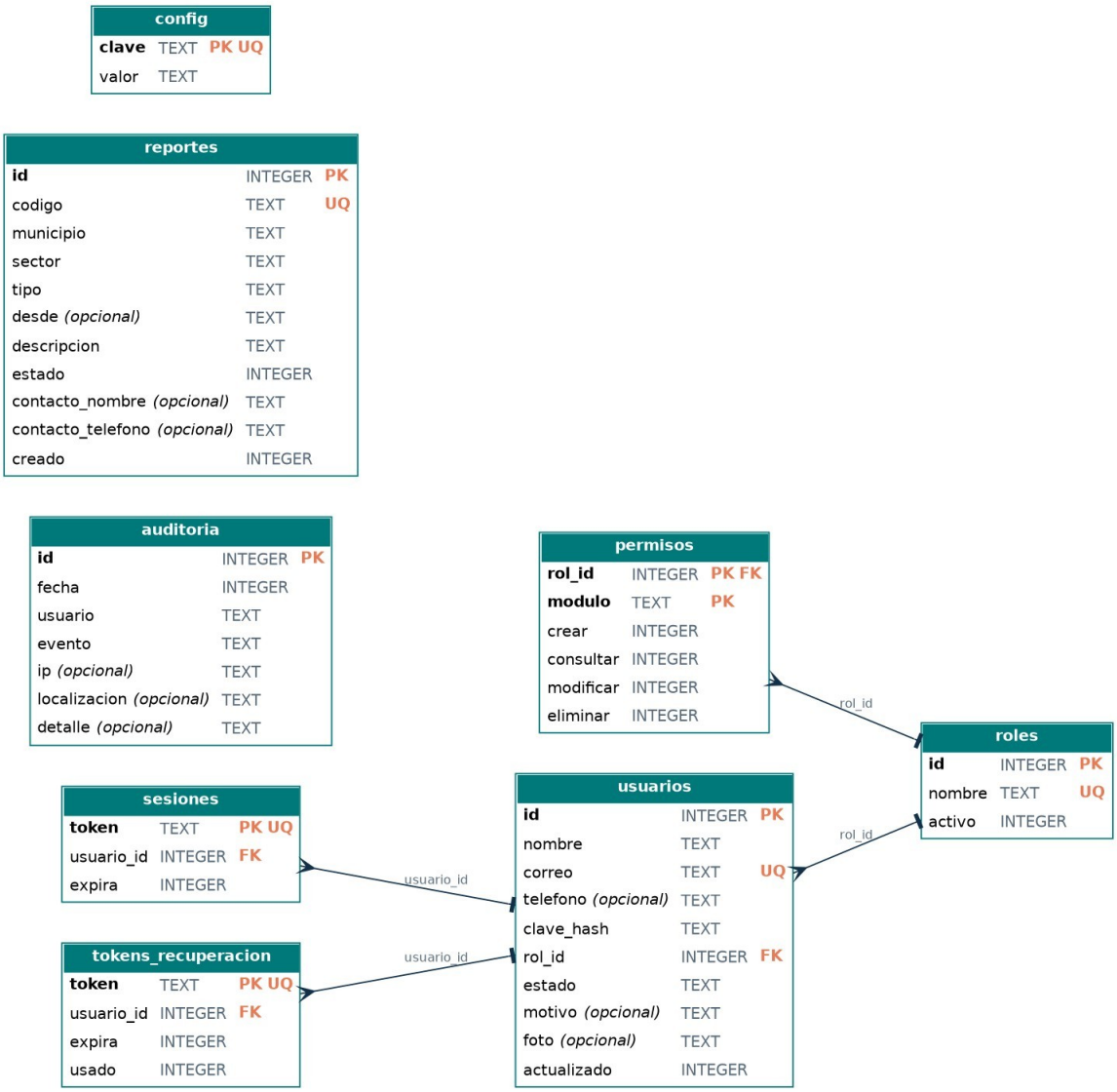
5.1 Diagramas entidad–relación

Figura 9. Vista general: las 16 tablas y sus relaciones



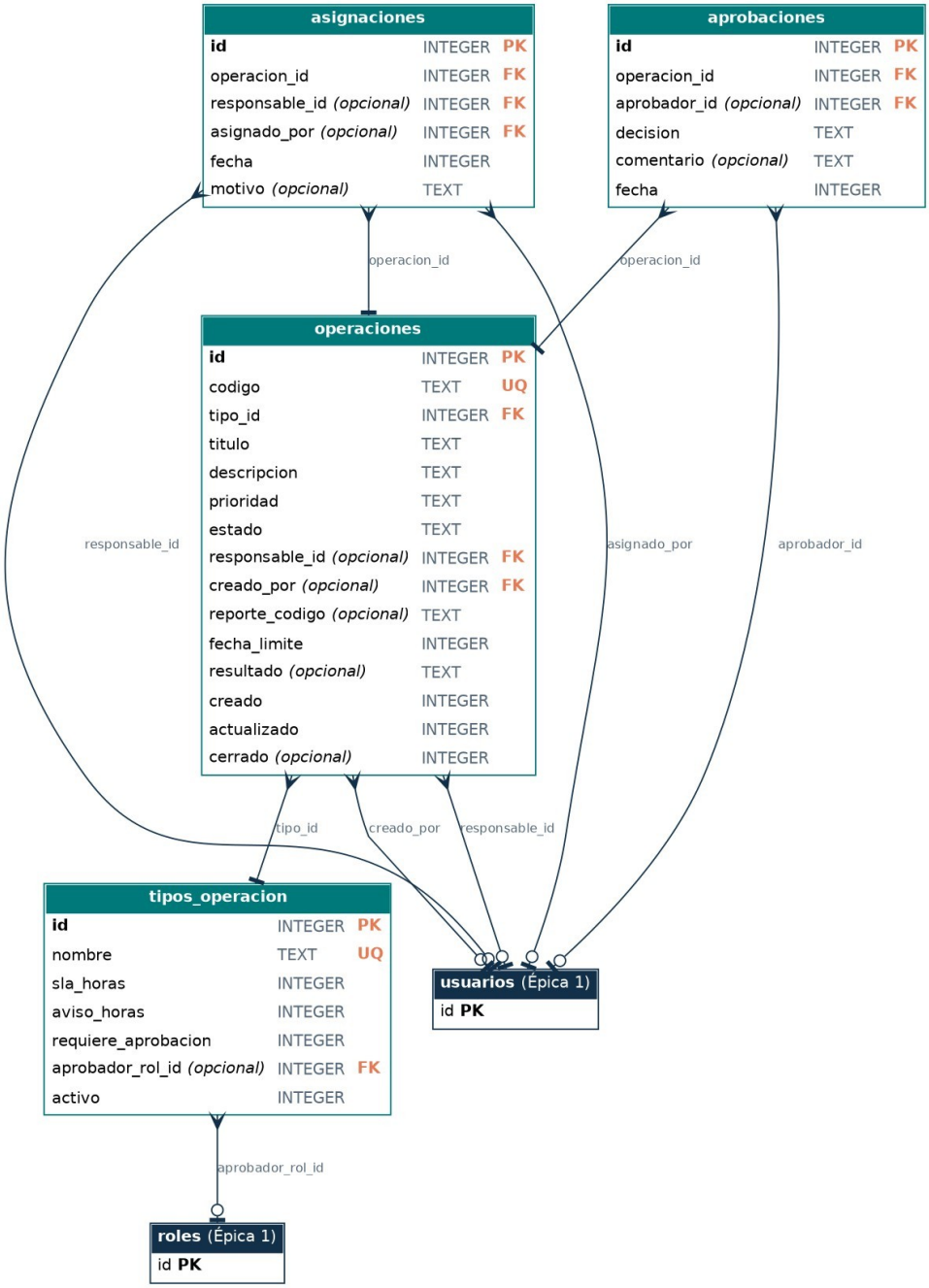
En verde, las tablas de la Épica 1; en azul, las de la Épica 2.

Figura 10. Épica 1: usuarios y accesos



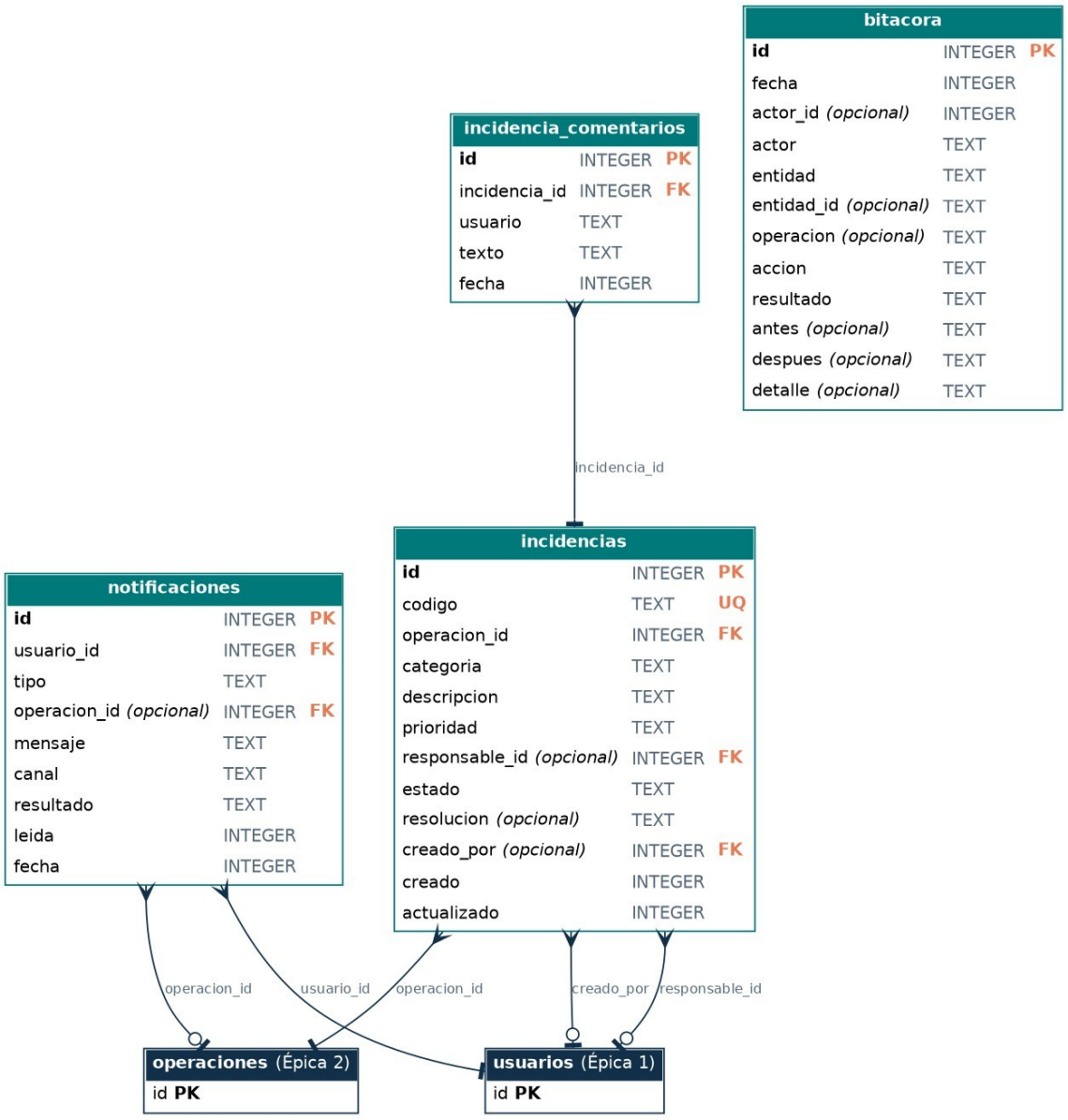
Un rol tiene muchos usuarios y muchos permisos (uno por módulo). Cada usuario puede tener muchas sesiones y muchos enlaces de recuperación. Auditoría, reportes y configuración son independientes.

Figura 11. Épica 2 (parte 1): operaciones, asignaciones y aprobaciones



Cada operación pertenece a un tipo; un tipo puede exigir que un rol apruebe. Las asignaciones y aprobaciones son el historial de cada operación.

Figura 12. Épica 2 (parte 2): incidencias, notificaciones y bitácora



Una operación tiene muchas incidencias, y cada incidencia muchos comentarios. La bitácora no tiene llaves foráneas a propósito (ver 5.3).

5.2 Modelo relacional y diccionario de datos

Cada tabla se describe con sus campos. PK = llave primaria, FK = llave foránea, UQ = valor único. Todos los campos son obligatorios salvo los marcados como opcionales.

roles

Campo	Tipo	Restricciones
id	INTEGER	PK

Campo	Tipo	Restricciones
nombre	TEXT	UQ · obligatorio
activo	INTEGER	obligatorio · por defecto 1

permisos

Campo	Tipo	Restricciones
rol_id	INTEGER	PK · FK → roles(id) · ON DELETE CASCADE
modulo	TEXT	PK
crear	INTEGER	obligatorio · por defecto 0
consultar	INTEGER	obligatorio · por defecto 0
modificar	INTEGER	obligatorio · por defecto 0
eliminar	INTEGER	obligatorio · por defecto 0

usuarios

Campo	Tipo	Restricciones
id	INTEGER	PK
nombre	TEXT	obligatorio
correo	TEXT	UQ · obligatorio
telefono	TEXT	opcional · por defecto ''
clave_hash	TEXT	obligatorio
rol_id	INTEGER	FK → roles(id) · obligatorio
estado	TEXT	obligatorio · por defecto 'Activo'
motivo	TEXT	opcional · por defecto ''
foto	TEXT	opcional
actualizado	INTEGER	obligatorio

sesiones

Campo	Tipo	Restricciones
token	TEXT	PK · UQ
usuario_id	INTEGER	FK → usuarios(id) · ON DELETE CASCADE · obligatorio
expira	INTEGER	obligatorio

tokens_recuperacion

Campo	Tipo	Restricciones
token	TEXT	PK · UQ
usuario_id	INTEGER	FK → usuarios(id) · ON DELETE CASCADE · obligatorio
expira	INTEGER	obligatorio
usado	INTEGER	obligatorio · por defecto 0

auditoria

Campo	Tipo	Restricciones
id	INTEGER	PK
fecha	INTEGER	obligatorio
usuario	TEXT	obligatorio
evento	TEXT	obligatorio
ip	TEXT	opcional
localizacion	TEXT	opcional
detalle	TEXT	opcional · por defecto ''

reportes

Campo	Tipo	Restricciones
id	INTEGER	PK
codigo	TEXT	UQ · obligatorio
municipio	TEXT	obligatorio
sector	TEXT	obligatorio
tipo	TEXT	obligatorio
desde	TEXT	opcional
descripcion	TEXT	obligatorio
estado	INTEGER	obligatorio · por defecto 0
contacto_nombre	TEXT	opcional
contacto_telefono	TEXT	opcional
creado	INTEGER	obligatorio

config

Campo	Tipo	Restricciones
clave	TEXT	PK · UQ
valor	TEXT	obligatorio

tipos_operacion

Campo	Tipo	Restricciones
id	INTEGER	PK
nombre	TEXT	UQ · obligatorio
sla_horas	INTEGER	obligatorio · por defecto 48
aviso_horas	INTEGER	obligatorio · por defecto 12
requiere_aprobacion	INTEGER	obligatorio · por defecto 0
aprobador_rol_id	INTEGER	FK → roles(id) · ON DELETE SET NULL · opcional
activo	INTEGER	obligatorio · por defecto 1

operaciones

Campo	Tipo	Restricciones
id	INTEGER	PK
codigo	TEXT	UQ · obligatorio
tipo_id	INTEGER	FK → tipos_operacion(id) · obligatorio
titulo	TEXT	obligatorio
descripcion	TEXT	obligatorio
prioridad	TEXT	obligatorio · por defecto 'Media'
estado	TEXT	obligatorio · por defecto 'Creada'
responsable_id	INTEGER	FK → usuarios(id) · ON DELETE SET NULL · opcional
creado_por	INTEGER	FK → usuarios(id) · ON DELETE SET NULL · opcional
reporte_codigo	TEXT	opcional
fecha_limite	INTEGER	obligatorio
resultado	TEXT	opcional
creado	INTEGER	obligatorio
actualizado	INTEGER	obligatorio

Campo	Tipo	Restricciones
cerrado	INTEGER	opcional

asignaciones

Campo	Tipo	Restricciones
id	INTEGER	PK
operacion_id	INTEGER	FK → operaciones(id) · ON DELETE CASCADE · obligatorio
responsable_id	INTEGER	FK → usuarios(id) · ON DELETE SET NULL · opcional
asignado_por	INTEGER	FK → usuarios(id) · ON DELETE SET NULL · opcional
fecha	INTEGER	obligatorio
motivo	TEXT	opcional · por defecto ''

aprobaciones

Campo	Tipo	Restricciones
id	INTEGER	PK
operacion_id	INTEGER	FK → operaciones(id) · ON DELETE CASCADE · obligatorio
aprobador_id	INTEGER	FK → usuarios(id) · ON DELETE SET NULL · opcional
decision	TEXT	obligatorio
comentario	TEXT	opcional · por defecto ''
fecha	INTEGER	obligatorio

incidencias

Campo	Tipo	Restricciones
id	INTEGER	PK
codigo	TEXT	UQ · obligatorio
operacion_id	INTEGER	FK → operaciones(id) · ON DELETE CASCADE · obligatorio
categoria	TEXT	obligatorio
descripcion	TEXT	obligatorio
prioridad	TEXT	obligatorio · por defecto 'Media'
responsable_id	INTEGER	FK → usuarios(id) · ON DELETE SET NULL · opcional
estado	TEXT	obligatorio · por defecto 'Abierta'
resolucion	TEXT	opcional

Campo	Tipo	Restricciones
creado_por	INTEGER	FK → usuarios(id) · ON DELETE SET NULL · opcional
creado	INTEGER	obligatorio
actualizado	INTEGER	obligatorio

incidencia_comentarios

Campo	Tipo	Restricciones
id	INTEGER	PK
incidencia_id	INTEGER	FK → incidencias(id) · ON DELETE CASCADE · obligatorio
usuario	TEXT	obligatorio
texto	TEXT	obligatorio
fecha	INTEGER	obligatorio

notificaciones

Campo	Tipo	Restricciones
id	INTEGER	PK
usuario_id	INTEGER	FK → usuarios(id) · ON DELETE CASCADE · obligatorio
tipo	TEXT	obligatorio
operacion_id	INTEGER	FK → operaciones(id) · ON DELETE CASCADE · opcional
mensaje	TEXT	obligatorio
canal	TEXT	obligatorio · por defecto 'interno'
resultado	TEXT	obligatorio · por defecto 'Entregada'
leida	INTEGER	obligatorio · por defecto 0
fecha	INTEGER	obligatorio

bitacora

Campo	Tipo	Restricciones
id	INTEGER	PK
fecha	INTEGER	obligatorio
actor_id	INTEGER	opcional
actor	TEXT	obligatorio
entidad	TEXT	obligatorio

Campo	Tipo	Restricciones
entidad_id	TEXT	opcional
operacion	TEXT	opcional
accion	TEXT	obligatorio
resultado	TEXT	obligatorio · por defecto 'Exitoso'
antes	TEXT	opcional
despues	TEXT	opcional
detalle	TEXT	opcional · por defecto ''

5.3 Normalización y decisiones de diseño

- **Tercera forma normal:** cada dato se guarda una sola vez; los atributos dependen de la llave de su tabla. Por ejemplo, el nombre del rol vive solo en roles y el usuario guarda rol_id.
- **Tablas de historial separadas:** asignaciones y aprobaciones guardan el historial de cada operación en lugar de sobrescribir el valor anterior.
- **Llave compuesta en permisos:** (rol_id, modulo) evita duplicados y expresa la matriz de permisos.
- **Bitácora sin llaves foráneas (desnormalización deliberada):** guarda el actor y la entidad como texto para conservar el historial aunque el usuario o la operación se eliminen, y para que no bloquee borrados.
- **Vínculo lógico operaciones → reportes:** operaciones.reporte_codigo guarda el código del reporte ciudadano sin llave foránea, porque los reportes ciudadanos son independientes y pueden depurarse sin afectar la operación.
- **Eliminación segura:** ON DELETE SET NULL en responsables y autores conserva las operaciones si se elimina un usuario; ON DELETE CASCADE elimina sesiones, tokens y permisos que dejan de tener sentido.
- **Integridad adicional:** los triggers bitacora_sin_modificar y bitacora_sin_eliminar rechazan cualquier UPDATE o DELETE sobre la bitácora, incluso ejecutado directamente en SQL.

5.4 Índices

Índice	Tabla	Para qué sirve
idx_auditoria_fecha	auditoria	Mostrar primero los accesos más recientes.
idx_op_estado	operaciones	Filtrar por estado en el panel.
idx_notif_usuario	notificaciones	Cargar las notificaciones sin leer de cada persona.
idx_bitacora_op	bitacora	Buscar el historial de una operación.
idx_bitacora_fecha	bitacora	Ordenar y filtrar por fecha.

6. Script de base de datos

El repositorio incluye tres scripts SQL en la carpeta `database/`:

Archivo	Contenido
`database/schema.sql`	Crea las 16 tablas, los 5 índices y los 2 triggers, con un comentario que explica cada tabla. Es idempotente (IF NOT EXISTS).
`database/seed.sql`	Datos iniciales mínimos: 9 roles, 108 permisos, 4 tipos de operación y los parámetros de configuración.
`database/seed_demo.sql`	Opcional: 9 usuarios de demostración (contraseña Demo123*, guardada con bcrypt) y un reporte de ejemplo.

6.1 Cómo ejecutarlos

Con la herramienta de línea de comandos de SQLite (o pegando el contenido en la pestaña Execute SQL de DB Browser for SQLite):

```
sqlite3 data/aguas-claras.db < database/schema.sql
sqlite3 data/aguas-claras.db < database/seed.sql
sqlite3 data/aguas-claras.db < database/seed_demo.sql  (opcional)
```

No es obligatorio ejecutarlos a mano: al iniciar con `npm start` la aplicación crea el mismo esquema automáticamente (server/db.js).

6.2 Verificación

Para demostrar que el script y la aplicación son equivalentes, se creó una base de datos nueva solo con los tres scripts y se comparó con la que crea la aplicación:

- Mismas **16 tablas**, con idénticos campos, tipos, llaves primarias y foráneas e índices.
- Mismos **2 triggers**: el UPDATE y el DELETE sobre la bitácora son rechazados con el mensaje “La bitácora no se puede modificar / eliminar”.
- Mismos datos iniciales: 9 roles, 108 permisos, 4 tipos de operación y 9 usuarios.
- La aplicación **arrancó sobre esa base** y el inicio de sesión y el panel de operaciones respondieron correctamente.

6.3 Extracto del script

```
-- operaciones: Trabajos operativos (código OP-0001) con estado, responsable, fecha límite y
resultado.
CREATE TABLE IF NOT EXISTS operaciones (
  id INTEGER PRIMARY KEY,
  codigo TEXT UNIQUE NOT NULL,
  tipo_id INTEGER NOT NULL REFERENCES tipos_operacion(id),
  titulo TEXT NOT NULL,
  descripcion TEXT NOT NULL,
  prioridad TEXT NOT NULL DEFAULT 'Media',
  estado TEXT NOT NULL DEFAULT 'Creada',
  responsable_id INTEGER REFERENCES usuarios(id) ON DELETE SET NULL,
```

```
  creado_por INTEGER REFERENCES usuarios(id) ON DELETE SET NULL,
  reporte_codigo TEXT,
  fecha_limite INTEGER NOT NULL,
  resultado TEXT,
  creado INTEGER NOT NULL,
  actualizado INTEGER NOT NULL,
  cerrado INTEGER
);
CREATE INDEX IF NOT EXISTS idx_op_estado ON operaciones(estado);

-- asignaciones: Historial de asignaciones y reasignaciones de responsables, con quién asignó
y el motivo.
CREATE TABLE IF NOT EXISTS asignaciones (
...
-- ---------- BITÁCORA INMUTABLE: bloqueos (triggers) ----------
-- Impiden modificar o eliminar cualquier registro de la bitácora, incluso desde SQL.
CREATE TRIGGER IF NOT EXISTS bitacora_sin_modificar BEFORE UPDATE ON bitacora BEGIN SELECT
RAISE(ABORT, 'La bitácora n
```

El script completo está en el repositorio (`database/schema.sql`, 210 líneas).

7. Configuración del entorno

7.1 Requisitos

Requisito	Detalle
Node.js	Versión 18 o superior (se desarrolló con la 22). Incluye npm.
Navegador	Chrome, Edge o Firefox actualizados.
Git	Para clonar el repositorio y guardar los cambios.
Opcional	DB Browser for SQLite para ver la base de datos.

7.2 Instalación y ejecución

1. Clonar el repositorio: `git clone <URL-del-repositorio>` y entrar con `cd aguas-claras`.
2. Instalar las dependencias: `npm install`.
3. Crear la configuración: `cp .env.example .env` (en Windows: `copy .env.example .env`).
4. Iniciar el sistema: `npm start`. La base de datos se crea sola.
5. Abrir http://localhost:3000 y entrar por Acceso con una cuenta de demostración (contraseña Demo123*).
6. Ejecutar las pruebas: `npm test` (resultado esperado: 63 aprobadas).

7.3 Variables de entorno (.env)

Variable	Para qué sirve
PORT	Puerto del servidor (3000 por defecto).

Variable	Para qué sirve
DB_FILE	Ruta del archivo de la base de datos.
SEED_DEMO	1 crea cuentas y datos de demostración; 0 para uso real.
ADMIN_EMAIL / ADMIN_PASSWORD	Super Administrador inicial cuando SEED_DEMO es 0.
MODO_DEMO	1 muestra el enlace de recuperación en pantalla (correo simulado).
COOKIE_SECURE / TRUST_PROXY	Activar (1) al publicar con HTTPS o detrás de un proxy.

El archivo `.env` y la carpeta `data/` están en `.gitignore`: nunca se suben al repositorio.

7.4 Estructura del repositorio

```
aguas-claras/
├── public/              front-end (index.html, css/, js/app.js,
js/operaciones.js)
├── server/              back-end (index.js, operaciones.js, db.js)
├── database/            scripts SQL (schema.sql, seed.sql, seed_demo.sql)
├── tests/               63 pruebas automáticas (api.test.js,
operaciones.test.js)
├── docs/                documentación y diagramas (docs/diagramas/)
├── evidencia/           videos y capturas de pantalla
├── package.json         dependencias y comandos (npm start, npm test)
├── .env.example         plantilla de configuración
└── README.md            instalación, arquitectura y endpoints
```

7.5 Control de versiones

- **Ramas:** main (versión estable) y una rama por incremento (feature/…), integradas con merge.
- **Commits:** mensajes descriptivos con prefijo (feat, fix, test, docs, refactor) y referencia a la historia de usuario.
- **Etiquetas:** una por entrega (v1.0, v2.0, v3.0).

8. Conclusiones y limitaciones

El diseño separa claramente presentación, aplicación y datos; concentra la seguridad y las reglas de negocio en el servidor; y deja un modelo de datos normalizado, con integridad y trazabilidad. Al estar implementado y probado, el diseño se validó con el comportamiento real del sistema, no solo en papel.

Limitaciones conocidas

- **Correo y mensajería simulados:** las alertas usan el canal interno de la página; el correo de recuperación se muestra en pantalla. Los servicios comerciales están evaluados en el apartado 3.3.

- **SQLite:** ideal para este proyecto; con muchos usuarios escribiendo a la vez conviene migrar a PostgreSQL o MySQL.
- **Alertas automáticas:** requieren el servidor encendido.
- **Localización por IP:** devuelve “Red local” o “No disponible” salvo que se instale geoip-lite.
- **Aprobación de un solo paso por tipo:** si se necesitan varios pasos en cadena, habría que ampliar el flujo.

Riesgos y mitigación

Riesgo	Mitigación
Pérdida de datos por borrar el archivo .db	Copiar la carpeta data/ periódicamente; los scripts permiten recrear el esquema.
Acceso no autorizado	Validación de sesión y permisos en el servidor, bloqueo por intentos y auditoría.
Cambios que rompan funciones existentes	63 pruebas automáticas antes de integrar.
Dependencia de servicios externos	Google Fonts es opcional; ningún servicio comercial es imprescindible hoy.

Anexo A. Catálogo de endpoints de la API

Extraído automáticamente del código fuente (server/index.js y server/operaciones.js): 55 endpoints.

Método	Ruta	Quién puede usarlo	Origen
POST	`/api/login`	Público	Épica 1 / público
POST	`/api/logout`	Público	Épica 1 / público
GET	`/api/me`	Público	Épica 1 / público
POST	`/api/acceso`	Sesión iniciada	Épica 1 / público
POST	`/api/probar`	Sesión iniciada	Épica 1 / público
POST	`/api/registro`	Público	Épica 1 / público
POST	`/api/usuarios`	Crear en usuarios	Épica 1 / público
GET	`/api/usuarios`	Consultar en usuarios	Épica 1 / público
PUT	`/api/usuarios/:id/rol`	Modificar en usuarios	Épica 1 / público
PUT	`/api/usuarios/:id/bloqueo`	Modificar en usuarios	Épica 1 / público
DELETE	`/api/usuarios/:id`	Eliminar en usuarios	Épica 1 / público
GET	`/api/roles`	Sesión iniciada	Épica 1 / público
POST	`/api/roles`	Crear en roles	Épica 1 / público
PUT	`/api/roles/:id`	Modificar en roles	Épica 1 / público

Método	Ruta	Quién puede usarlo	Origen
PUT	`/api/roles/:id/estado`	Modificar en roles	Épica 1 / público
DELETE	`/api/roles/:id`	Eliminar en roles	Épica 1 / público
GET	`/api/permisos`	Sesión iniciada	Épica 1 / público
PUT	`/api/permisos`	Modificar en permisos	Épica 1 / público
GET	`/api/auditoria`	Consultar en auditoria	Épica 1 / público
GET	`/api/config`	Sesión iniciada	Épica 1 / público
PUT	`/api/config`	Modificar en usuarios	Épica 1 / público
PUT	`/api/perfil`	Sesión iniciada	Épica 1 / público
PUT	`/api/perfil/clave`	Sesión iniciada	Épica 1 / público
POST	`/api/recuperar`	Público	Épica 1 / público
POST	`/api/recuperar/usar`	Público	Épica 1 / público
POST	`/api/reportes`	Público	Épica 1 / público
GET	`/api/reportes/publico/:codigo`	Público	Épica 1 / público
GET	`/api/reportes`	Consultar en reportes	Épica 1 / público
PUT	`/api/reportes/:codigo/estado`	Modificar en reportes	Épica 1 / público
DELETE	`/api/reportes/:codigo`	Eliminar en reportes	Épica 1 / público
GET	`/api/operaciones`	Consultar en operaciones	Épica 2
GET	`/api/responsables`	Consultar en operaciones	Épica 2
GET	`/api/operaciones/:codigo`	Consultar en operaciones	Épica 2
POST	`/api/operaciones`	Crear en operaciones	Épica 2
PUT	`/api/operaciones/:codigo`	Modificar en operaciones	Épica 2
POST	`/api/operaciones/:codigo/estado`	Modificar en operaciones	Épica 2
POST	`/api/operaciones/:codigo/asignar`	Crear en asignaciones	Épica 2
POST	`/api/operaciones/:codigo/aprobacion`	Modificar en aprobaciones	Épica 2
GET	`/api/aprobaciones`	Consultar en aprobaciones	Épica 2
GET	`/api/tipos`	Sesión iniciada	Épica 2
POST	`/api/tipos`	Crear en tipos	Épica 2
PUT	`/api/tipos/:id`	Modificar en tipos	Épica 2

Método	Ruta	Quién puede usarlo	Origen
POST	`/api/alertas/generar`	Modificar en operaciones	Épica 2
GET	`/api/alertas`	Consultar en operaciones	Épica 2
PUT	`/api/alertas/config`	Modificar en tipos	Épica 2
GET	`/api/notificaciones`	Sesión iniciada	Épica 2
PUT	`/api/notificaciones/leidas`	Sesión iniciada	Épica 2
GET	`/api/incidencias`	Consultar en incidencias	Épica 2
GET	`/api/incidencias/:codigo`	Consultar en incidencias	Épica 2
POST	`/api/incidencias`	Crear en incidencias	Épica 2
POST	`/api/incidencias/:codigo/estado`	Modificar en incidencias	Épica 2
POST	`/api/incidencias/:codigo/comentarios`	Modificar en incidencias	Épica 2
GET	`/api/informes`	Consultar en informes	Épica 2
GET	`/api/informes/exportar`	Crear en informes	Épica 2
GET	`/api/bitacora`	Consultar en bitacora	Épica 2

Anexo B. Cómo cumple este documento la rúbrica

Criterio	Puntos	Dónde se cumple
Arquitectura y diagramas	3	Apartado 4: arquitectura en tres capas, despliegue, estructura del front-end y del back-end, tres diagramas de secuencia y el diagrama de estados (figuras 1 a 9), con la explicación de cada uno.
Modelo relacional y script de BD	2	Apartados 5 y 6: cuatro diagramas entidad–relación, diccionario de datos de las 16 tablas, normalización, índices, triggers y los scripts schema.sql, seed.sql y seed_demo.sql, verificados contra la aplicación.

Instrucciones del entregable: (1) requerimientos funcionales y no funcionales → apartado 2; (2) tecnologías open source y propietarias → apartado 3; (3) diseño de arquitectura con estructura del front-end, del back-end y modelo de interacción → apartado 4.

Entrega: este documento, el repositorio en GitHub y la presentación en video del diseño.